Day 38 : Introduction To React

Introduction to React

React is a JavaScript library for building user interfaces. But why do we need it? Let's understand from first principles.

Problem Statement

Task: Create multiple HTML elements with styles, className, id, and text content using JavaScript.

Without any helper (Traditional way):

```
// Create first element
const h1 = document.createElement('h1');
h1.style.backgroundColor = 'orange';
h1.style.color = 'black';
h1.style.fontSize = '30px';
h1.id = 'first';
h1.className = 'ele1';
h1.textContent = 'Hello Coder Army';
document.getElementById('root').appendChild(h1);

// Create second element
const p = document.createElement('p');
p.style.color = 'blue';
p.id = 'para';
p.className = 'text';
p.textContent = 'This is a paragraph';
document.getElementById('root').appendChild(p);
```

```
// Create third element... (repetitive!)
```

Problems:

1. Repetitive code
2. Hard to maintain
3. Error-prone
4. Not reusable

Solution 1: Create Our Own Helper Library

Let's create a simple library to make element creation easier:

```
const React = {
  createElement: function(tag, attribute, children){
    const element = document.createElement(tag);

    for(const key in attribute){
      if(key === 'style'){
        Object.assign(element.style, attribute[key]);
      }
      else {
        element[key] = attribute[key];
      }
    }

    element.textContent = children;
    return element;
  }
};

const ReactDOM = {
  render: function(element, root){
    root.appendChild(element);
```

```
    }
};
```

Usage:

```
const element1 = React.createElement(
    'h1',
    {
        style: {backgroundColor: "orange", color: "black", fontSize: "30px"},
        id: "first",
        className: "ele1"
    },
    "Hello Coder Army"
);

ReactDOM.render(element1, document.getElementById('root'));
```

Benefits:

- ✅ Cleaner code
- ✅ Reusable
- ✅ Easy to create multiple elements

Moving to Real React

Our library works, but let's use the actual React library via CDN:

```
<!DOCTYPE html>
<html>
<head>
    <title>React App</title>
</head>
<body>
    <div id="root"></div>
```

```
    <!-- React CDN -->
    <script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

    <script>
        const element = React.createElement(
            'h1',
            {
                style: {backgroundColor: "orange", color: "black", fontSize: "30px"},
                id: "first",
                className: "ele1"
            },
            "Hello Coder Army"
        );

        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(element);
    </script>
</body>
</html>
```

Notice: The code looks similar, but there are differences in how React actually works internally.

How React Actually Works (Deep Dive)

Key Concept: Virtual DOM

Our library directly creates real DOM elements. But **real React does something different**.

What React.createElement Actually Returns:

```
const React = {
   createElement: function(type, props, children){
      return {  // Returns a plain JavaScript object (Virtual DOM)
         type: type,
         props: {
            ...props,
            children: children
         }
      };
   }
};
```

Example:

```
const element = React.createElement('h1', { id: 'title' }, 'Hello');

console.log(element);
// Output: (Just a JavaScript object!)
// {
//    type: 'h1',
//    props: {
//       id: 'title',
//       children: 'Hello'
//    }
// }
```

This object is called Virtual DOM - it's NOT a real DOM element, just a description of what we want.

What ReactDOM Actually Does:

ReactDOM takes the Virtual DOM object and converts it to real DOM:

```
const ReactDOM = {
   createRoot: function(container){
      return {
         render: function(reactElement){
            // Convert Virtual DOM to Real DOM
            const element = document.createElement(reactElement.type);

            for(const key in reactElement.props){
               if(key === 'style'){
                  Object.assign(element.style, reactElement.props.style);
               }
               else if(key === 'children'){
                  element.textContent = reactElement.props[key];
               }
               else {
                  element[key] = reactElement.props[key];
               }
            }

            container.appendChild(element);
         }
      };
   }
};
```

First Principle: Why Separate React and ReactDOM?

Question: Why two separate libraries?

Answer: Because React creates platform-agnostic descriptions (Virtual DOM), and different renderers convert them to platform-specific UI.

The Flow:

```
Your Code
   ↓
React.createElement() → Creates Virtual DOM (JavaScript object)
   ↓
Virtual DOM (works everywhere - web, mobile, PDF, VR)
   ↓
   ├─→ ReactDOM → Browser (HTML elements)
   ├─→ React Native → Mobile (iOS/Android components)
   ├─→ React PDF → PDF documents
   └─→ React VR → Virtual Reality
```

Example: Same Virtual DOM, Different Platforms

The Virtual DOM (same everywhere):

```
const vdom = {
  type: 'View',
  props: {
    style: { padding: 20 },
    children: [
      { type: 'Text', props: { children: 'Hello World' } },
      { type: 'Button', props: { onPress: handleClick, children: 'Click' } }
    ]
  }
};
```

ReactDOM (Web) converts to:

```
<div style="padding: 20px">
  <span>Hello World</span>
  <button onclick="handleClick()">Click</button>
</div>
```

React Native (Mobile) converts to:

```
UIView (padding: 20)
    ├─ UILabel: "Hello World"
    └─ UIButton: "Click" (onPress: handleClick)
```

React PDF converts to:

```
PDF Page
    ├─ Text: "Hello World" at (x: 20, y: 20)
    └─ Rectangle: "Click" at (x: 20, y: 50)
```

Same Virtual DOM, different outputs!

Why This Design?

Without Separation (Bad):

```
// Would need separate code for each platform
ReactWeb.createElement('button', ...);      // For web
ReactMobile.createElement('button', ...);   // For mobile
ReactPDF.createElement('button', ...);      // For PDF
```

With Separation (Good):

```
// Same code works everywhere
React.createElement('button', ...);  // Platform-agnostic

// Different renderers handle conversion
ReactDOM.render(...);       // Web
ReactNative.render(...);    // Mobile
ReactPDF.render(...);       // PDF
```

React 18: createRoot vs render

Old Way (React 17):

```
ReactDOM.render(<App />, document.getElementById('root'));
```

New Way (React 18):

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Why Separate?

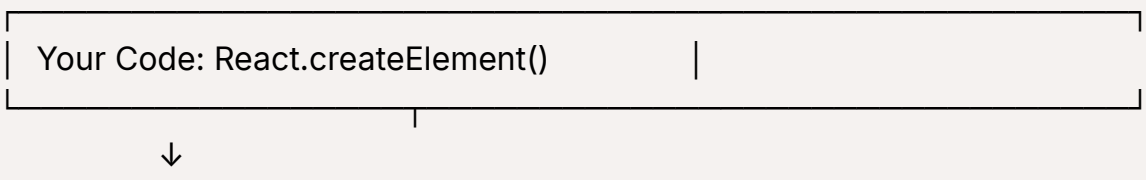
First Principle: Separate "setup" from "action"

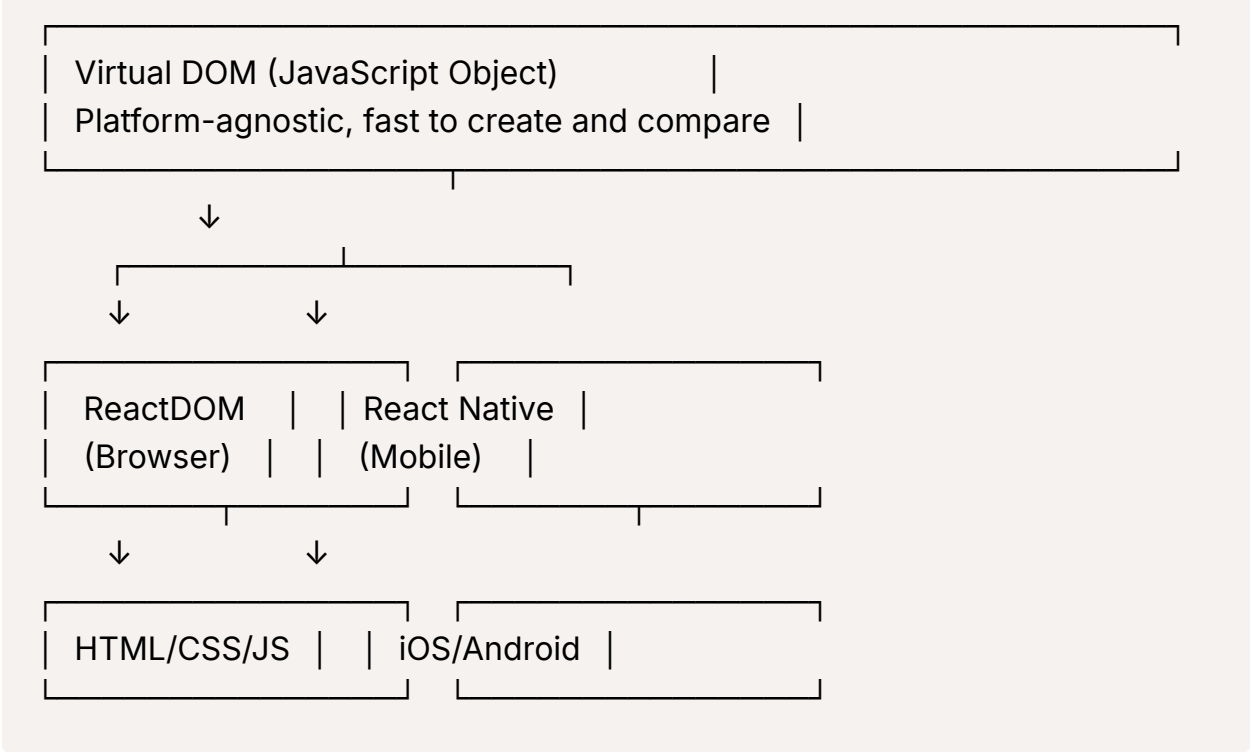
```
// Setup (expensive, done ONCE)
const root = ReactDOM.createRoot(container);
// - Find container
// - Setup event system
// - Initialize fiber tree
// - Configure rendering mode

// Render (cheap, done MANY times)
root.render(<App />);
root.render(<App />);
root.render(<App />);
```

Analogy: Opening a bank account (once) vs making transactions (many times).

Summary Diagram





React is the engine, renderers are the wheels for different terrains!

React: It is a js Library

React's Job: The "What" - Its job is only to describe what the UI should look like. It doesn't create DOM elements. It creates lightweight JavaScript objects that act as blueprints.

→

```
{
type: 'h1"
props:{
style:{},
children:{},
className:
id:
}
```

ReactDOM's Job: The "How" - This is the renderer. Its job is to take the blueprint from React and actually build the UI for a specific platform (in this case, a web browser's DOM).

→ document.createElement()